Mohaiya aldossary
Student ID: 2230004579

In [1]:
```
%pip install -q pandas scikit-learn numpy gensim torch optree==0.20.0
```

```
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 23.2.1 -> 26.2.1
[notice] To update, run: C:\Users\ezio9\OneDrive\Desktop\IAU\NLP\NLP lab\.lab_work\venv\Scripts\python.exe -m pip install --upgrade pip
```

Deep Learning for NLP

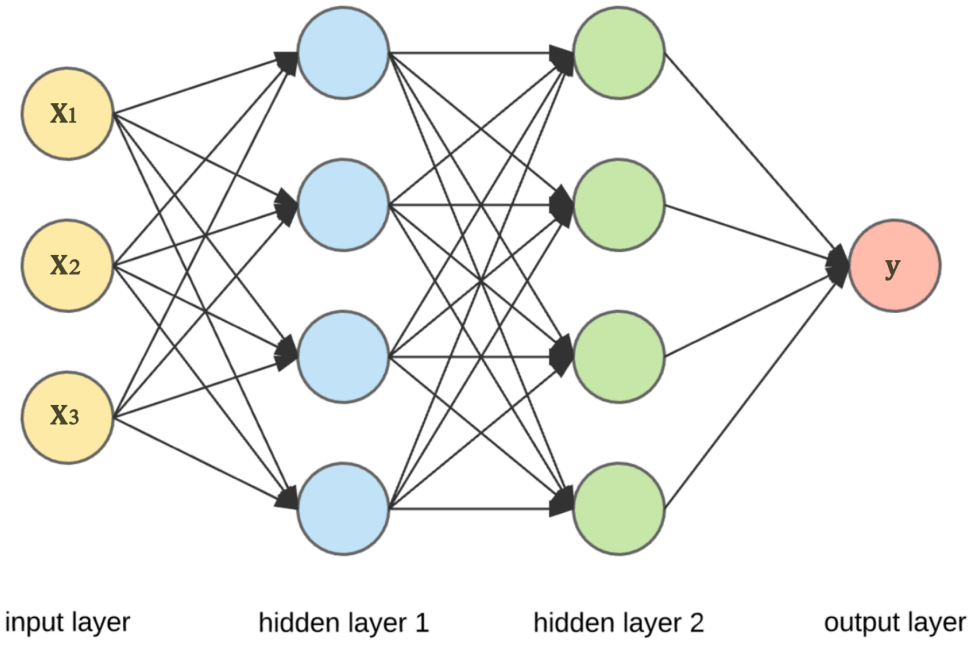
Deep Learning is a type of Machine Learning that uses **Neural Networks with multiple layers** to learn patterns from data.

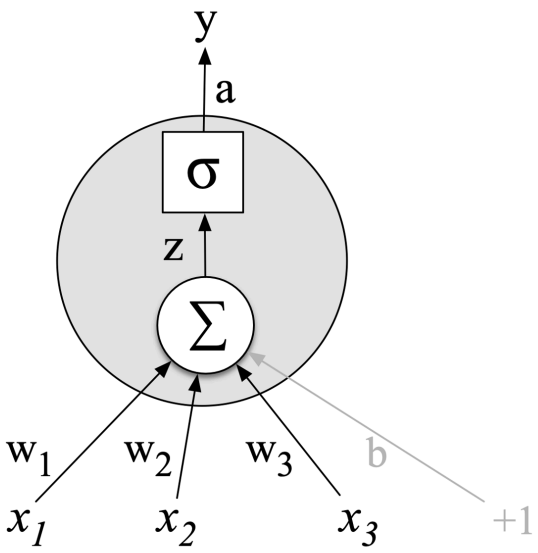
Deep Learning can be used in NLP tasks such as:

- Text Classification
- Sentiment Analysis
- Question Answering

In this lab, we will build a **deep learning model** to classify Yelp restaurant reviews as:

- **0 = Negative**
- **1 = Positive**





1. Load the Yelp Dataset

Each row contains:

- a restaurant review
- its sentiment label

In [2]:
```
import pandas as pd

df = pd.read_csv("07-yelp-dataset.txt", sep="\t", names=["review", "label"])

print("Number of reviews:", len(df))
df.head()
```

```
Number of reviews: 1000
```

Out[2]:

	review	label
0	Wow... Loved this place.	1
1	Crust is not good.	0
2	Not tasty and the texture was just nasty.	0
3	Stopped by during the late May bank holiday of...	1
4	The selection on the menu was great and so wer...	1

In [3]:
```
print(df["label"].value_counts())
```

```
label
1    500
0    500
Name: count, dtype: int64
```

2. Preprocessing the Dataset

In [4]:

```
import re

def preprocess_text(text):

    # 1. Convert to lowercase
    text = text.lower()

    # 2. Remove punctuation and special characters
    text = re.sub(r'[^a-zA-Z\s]', '', text)

    # 3. Remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()

    return text

df["review"] = df["review"].apply(preprocess_text)
df.head()
```

Out[4]:

	review	label
0	wow loved this place	1
1	crust is not good	0
2	not tasty and the texture was just nasty	0
3	stopped by during the late may bank holiday of...	1
4	the selection on the menu was great and so wer...	1

3. Split the Data

We divide the dataset into:

- **80% training data** — used to train the model
- **20% testing data** — used to evaluate the model

`stratify=y` keeps the proportion of positive and negative reviews similar in both sets.

In [5]:

```
from sklearn.model_selection import train_test_split

X = df["review"]
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

print("Training reviews:", len(X_train))
print("Testing reviews:", len(X_test))
```

```
Training reviews: 800
Testing reviews: 200
```

4. Convert Text to TF-IDF Vectors

A neural network cannot directly understand text.

TF-IDF converts each review into a vector of numbers.

In [6]:

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(max_features=1000, lowercase=True)

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("Training TF-IDF shape:", X_train_tfidf.shape)
print("Testing TF-IDF shape:", X_test_tfidf.shape)
```

```
Training TF-IDF shape: (800, 1000)
Testing TF-IDF shape: (200, 1000)
```

5. Convert the Data to PyTorch Tensors

PyTorch neural networks work with **tensors**.

In [7]:

```
import torch
import torch.nn as nn

# Convert TF-IDF data to PyTorch tensors
X_train_tensor = torch.FloatTensor(X_train_tfidf.toarray())
X_test_tensor = torch.FloatTensor(X_test_tfidf.toarray())

y_train_tensor = torch.FloatTensor(y_train.values).reshape(-1, 1)
```

```
C:\Users\ezio9\AppData\Local\Temp\ipykernel_6400\803754565.py:8: UserWarning: The given NumPy array is not writable, and PyTorch does not support non-writable tensors. This means writing to this tensor will result in undefined behavior. You may want to copy the array to protect its data or make it writable before converting it to a tensor. This type of warning will be suppressed for the rest of this program. (Triggered internally at C:\actions-runner\_work\pytorch\pytorch\builder\windows\pytorch\torch\csrc\utils\tensor_numpy.cpp:212.)
  y_train_tensor = torch.FloatTensor(y_train.values).reshape(-1, 1)
```

6. Build a Simple Neural Network

Our network has:

1. **Input layer** — receives the TF-IDF features
2. **Hidden layer** — learns useful patterns
3. **ReLU activation** — adds non-linearity
4. **Output layer** — produces one value for binary classification

We do **not** put `Sigmoid` inside the model because `BCEWithLogitsLoss` applies it internally in a numerically stable way.

In [8]:

```
input_size = X_train_tensor.shape[1]
# torch.size([800, 1000])

model = nn.Sequential(
    nn.Linear(input_size, 64),   # Input layer
    nn.ReLU(),

    nn.Linear(64, 32),           # Hidden layer
    nn.ReLU(),

    nn.Linear(32, 1)             # Output layer
)

print(model)
```

```
Sequential(
  (0): Linear(in_features=1000, out_features=64, bias=True)
  (1): ReLU()
  (2): Linear(in_features=64, out_features=32, bias=True)
  (3): ReLU()
  (4): Linear(in_features=32, out_features=1, bias=True)
)
```

7. Choose the Loss Function and Optimizer

- **BCEWithLogitsLoss** is used for binary classification. To measure the model's mistakes.
- **Adam** is responsible for changing the model's weights so that the loss becomes smaller.

In [9]:

```
loss_function = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

8. Train the Neural Network

One **epoch** means the model has seen the full training dataset once.

For every epoch:

1. Make predictions
2. Calculate the loss
3. Calculate gradients with backpropagation
4. Update the weights

In [10]:

```
epochs = 10

for epoch in range(epochs):
    model.train()
    optimizer.zero_grad()

    # 1. Forward pass
    outputs = model(X_train_tensor)

    # 2. Calculate loss
    loss = loss_function(outputs, y_train_tensor)

    # 3. Backpropagation
    loss.backward()

    # 4. Update model weights
    optimizer.step()

    if (epoch + 1) % 5 == 0:
        print(f"Epoch {epoch + 1}/{epochs} - Loss: {loss.item():.4f}")
```

```
Epoch 5/10 - Loss: 0.6507
Epoch 10/10 - Loss: 0.4545
```

9. Evaluate the Model

The model outputs raw scores called **logits**.

We apply `sigmoid` to convert them to probabilities between 0 and 1.

- Probability ≥ **0.5** → **Positive (1)**
- Probability < **0.5** → **Negative (0)**

In [11]:

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

model.eval()

with torch.no_grad():
    test = model(X_test_tensor)
    test_probabilities = torch.sigmoid(test)
    predictions = (test_probabilities >= 0.5).int().numpy().flatten()

accuracy = accuracy_score(y_test, predictions)

print(f"Test Accuracy: {accuracy:.3f}")
```

```
Test Accuracy: 0.765
```

10. Classification Report and Confusion Matrix

The classification report shows:

- **Precision**
- **Recall**
- **F1-score**

The confusion matrix shows how many reviews were classified correctly and incorrectly.

In [12]:

```
print("Classification Report:")
print(classification_report(y_test, predictions, target_names=["Negative", "Positive"]))

print("Confusion Matrix:")
print(confusion_matrix(y_test, predictions))
```

```
Classification Report:
              precision    recall  f1-score   support

    Negative       0.71      0.86      0.78        96
    Positive       0.84      0.67      0.75       104

    accuracy                           0.77       200
   macro avg       0.78      0.77      0.76       200
weighted avg       0.78      0.77      0.76       200

Confusion Matrix:
[[83 13]
 [34 70]]
```

Task 1 – Represent Reviews Using Word2Vec

In the previous part of the lab, TF-IDF was used to convert each review into numerical features. Now, use Word2Vec instead.

Requirements: Tokenize the training and testing reviews. Train a Word2Vec model using the training reviews. Use: vector_size = 200 window = 5 min_count = 1 sg = 1

Convert each review into a single vector. Convert the resulting vectors into PyTorch tensors.

In [13]:

```
import numpy as np
from gensim.models import Word2Vec

train_tokens = [review.split() for review in X_train]
test_tokens = [review.split() for review in X_test]
word2vec = Word2Vec(
    sentences=train_tokens, vector_size=200, window=5, min_count=1, sg=1,
    seed=42, workers=1
)

def review_vector(tokens):
    vectors = [word2vec.wv[word] for word in tokens if word in word2vec.wv]
    return np.mean(vectors, axis=0) if vectors else np.zeros(200, dtype=np.float32)

X_train_w2v = torch.tensor(np.array([review_vector(tokens) for tokens in train_tokens]), dtype=torch.float32)
X_test_w2v = torch.tensor(np.array([review_vector(tokens) for tokens in test_tokens]), dtype=torch.float32)
print('Training Word2Vec shape:', X_train_w2v.shape)
print('Testing Word2Vec shape:', X_test_w2v.shape)
```

```
Training Word2Vec shape: torch.Size([800, 200])
Testing Word2Vec shape: torch.Size([200, 200])
```

Task 2 – Build a Different Neural Network

Use the Word2Vec vectors from Task 1 as the input to a new neural network. Your network must contain: An input layer Three hidden layers ReLU activation functions One output neuron Use the following architecture:

Input

↓

128 neurons

↓

64 neurons

↓

32 neurons

↓

1 output neuron

Train the model using: , BCEWithLogitsLoss , Adam optimizer , Learning rate = 0.001 , 15 epochs. Then calculate the test accuracy.

In [14]:

```
torch.manual_seed(42)
w2v_model = nn.Sequential(
    nn.Linear(200, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Linear(32, 1)
)
print(w2v_model)
w2v_loss_function = nn.BCEWithLogitsLoss()
w2v_optimizer = torch.optim.Adam(w2v_model.parameters(), lr=0.001)

for epoch in range(15):
    w2v_model.train()
    w2v_optimizer.zero_grad()
    logits = w2v_model(X_train_w2v)
    loss = w2v_loss_function(logits, y_train_tensor)
    loss.backward()
    w2v_optimizer.step()
    print(f'Epoch {epoch + 1}/15 - Loss: {loss.item():.4f}')

w2v_model.eval()
with torch.no_grad():
    w2v_predictions = (torch.sigmoid(w2v_model(X_test_w2v)) >= 0.5).int().numpy().flatten()
print(f'Test Accuracy: {accuracy_score(y_test, w2v_predictions):.3f}')
```

```
Sequential(
  (0): Linear(in_features=200, out_features=128, bias=True)
  (1): ReLU()
  (2): Linear(in_features=128, out_features=64, bias=True)
  (3): ReLU()
  (4): Linear(in_features=64, out_features=32, bias=True)
  (5): ReLU()
  (6): Linear(in_features=32, out_features=1, bias=True)
)
Epoch 1/15 - Loss: 0.6965
Epoch 2/15 - Loss: 0.6959
Epoch 3/15 - Loss: 0.6955
Epoch 4/15 - Loss: 0.6951
Epoch 5/15 - Loss: 0.6947
Epoch 6/15 - Loss: 0.6944
Epoch 7/15 - Loss: 0.6941
Epoch 8/15 - Loss: 0.6939
Epoch 9/15 - Loss: 0.6936
Epoch 10/15 - Loss: 0.6935
Epoch 11/15 - Loss: 0.6934
Epoch 12/15 - Loss: 0.6933
Epoch 13/15 - Loss: 0.6932
Epoch 14/15 - Loss: 0.6932
Epoch 15/15 - Loss: 0.6933
Test Accuracy: 0.480
```